

Machine Learning Project

Auto MPG

Machine Learning Pipeline

Previsione del consumo carburante con regressione lineare

Andrea Morlotti · Riccardo Scampini

UCI Auto MPG
398 righe · 7 feature

Obiettivo del Progetto

Data le caratteristiche tecniche di un'auto, quanto consuma?

Dataset reale

Auto MPG dell'UCI — 398 auto degli anni '70-'80 con caratteristiche tecniche e consumo misurato in miglia per gallone (mpg)

Pipeline completa

Dal dato grezzo al modello addestrato: pulizia, selezione feature, split, scaling, regressione

API REST

Ogni step della pipeline esposto come endpoint HTTP tramite Flask, containerizzato con Docker

Valutazione rigorosa

Pipeline corretta senza data leakage: split prima dello scaling, metriche su test set mai visto

Il Dataset — Auto MPG

398

Righe

auto anni 70-82

7

Feature

dopo pulizia

mpg

Target

miglia per gallone

6

Mancanti

su horsepower (1.5%)

Colonne nel dataset originale

car_name Etichetta testuale — non informativa rimossa

cylinders Numero di cilindri (4, 6, 8)

displacement Cilindrata in centimetri cubici

horsepower Potenza in cavalli — ha 6 mancanti

weight Peso del veicolo in libbre

acceleration Secondi per raggiungere 60 mph

model_year Anno di produzione (70-82)

origin 1=USA 2=Europa 3=Giappone

mpg TARGET — consumo da prevedere

Analisi Esplorativa (EDA)

Prima di costruire il modello abbiamo analizzato i dati per capire distribuzione, outlier e anomalie.

Statistiche Descrittive

Media, mediana, deviazione standard, min/max per ogni feature — endpoint GET /eda/info

Outlier con IQR

Identifica valori anomali

$IQR = Q_3 - Q_1$
Outlier se valore $< Q_1 - 1.5 \times IQR$ oppure valore $> Q_3 + 1.5 \times IQR$
— GET /eda/outliers/<col>

Test di Normalità

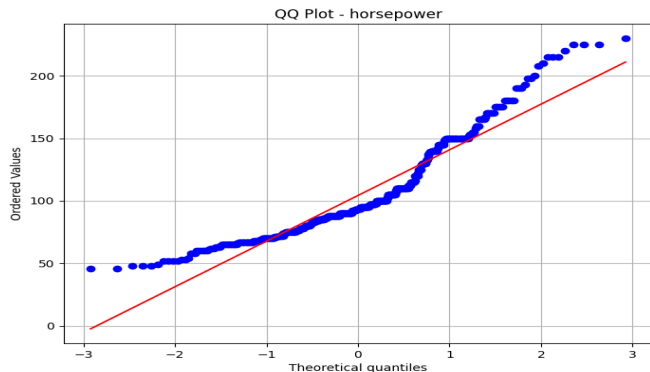
Jarque-Bera e D'Agostino-Pearson testano se i dati seguono distribuzione normale — utile per scegliere strategia sui mancanti

Grafici Visivi

QQ-plot: deviazione dalla normalità. Boxplot: outlier.
Istogrammi: distribuzione di ogni feature — PNG via API

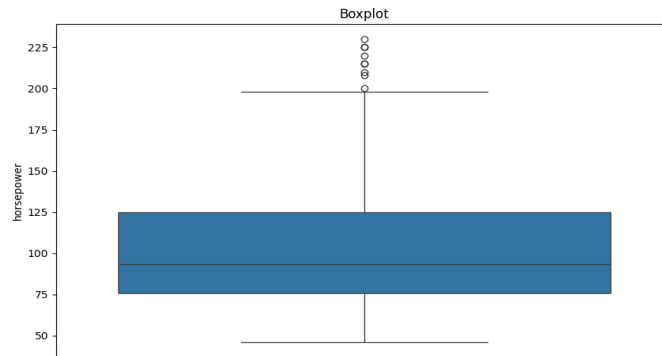
EDA — Analisi Visiva su horsepower

QQ-Plot



I punti si discostano dalla linea rossa → distribuzione NON normale

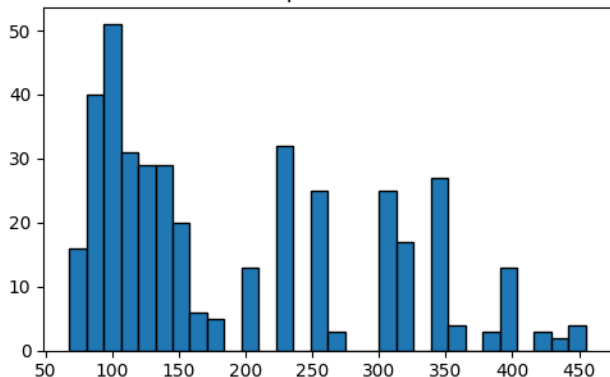
Boxplot



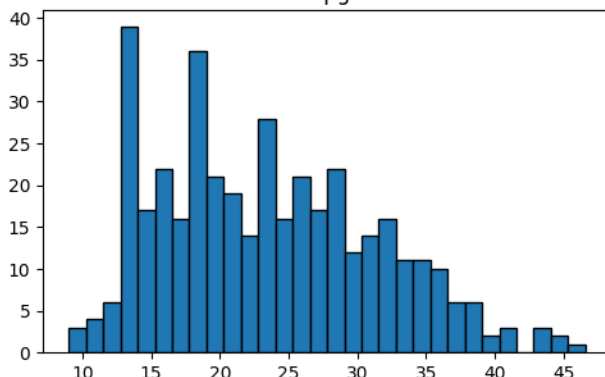
Whisker corto + cerchi in alto → outlier elevati confermati

Istogrammi delle Features

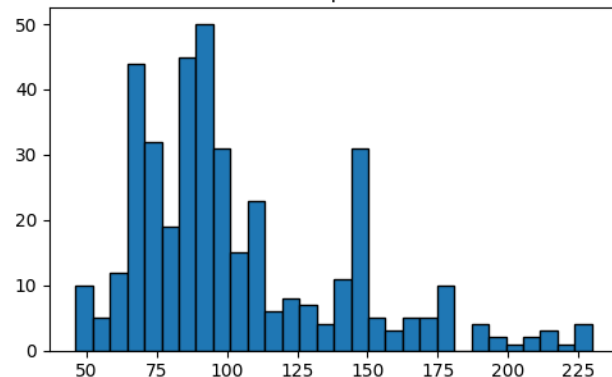
displacement



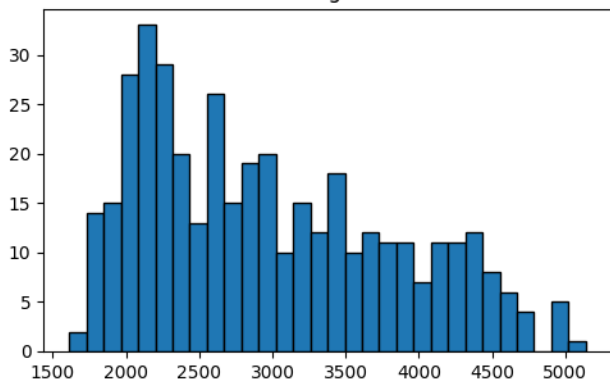
mpg



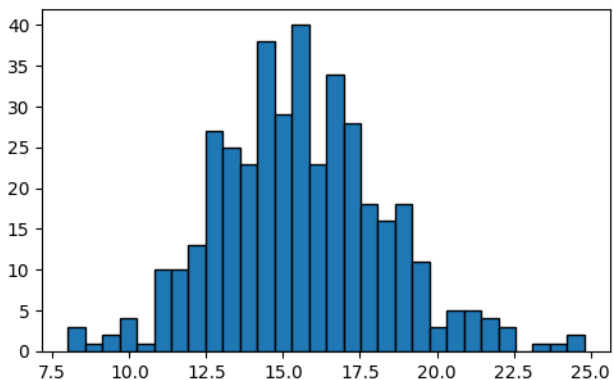
horsepower



weight



acceleration



Data Cleaning — Valori Mancanti

Il Problema

horsepower contiene 6 valori mancanti (originariamente '?' nel file UCI).

Il DataLoader li converte in NaN, DataCleaner li gestisce con la mediana.

*Solo 1.51% del totale
→ la colonna NON va eliminata*

Horsepower ha outlier → mediana = 93.50

Situazione	Strategia
Distribuzione normale	Media
Outlier presenti ✓	Mediana ✓
Variabile categorica	Moda

Feature Selection — Variance Threshold

Una feature con varianza quasi zero non porta informazione — tutte le righe hanno lo stesso valore.

Come funziona

1. Calcola la varianza di ogni feature
2. **soglia = $p \times \text{varianza_massima}$**
3. Rimuove le feature sotto soglia

Parametro usato: $p = 0.75$

Risultato nel progetto

Feature rimosse: nessuna ✓

Tutte e 7 le feature hanno
varianza
sufficiente per essere
mantenute.

displacement

horsepower

acceleration

origin

cylinders

weight

model_year

Input: 8 colonne dopo DataLoader → Varianza calcolata → Output: 7 feature mantenute

Train / Test Split

Per valutare correttamente il modello bisogna testarlo su dati che non ha mai visto durante l'addestramento.

TRAIN — 318 righe (80%)

**TEST
80 righe (20%)**

398 righe totali

Cosa va nel TRAIN

- Il modello impara i pattern dal train set
- Lo scaler calcola media e std sul train
- Usato per minimizzare l'errore (fit)
- Non deve essere troppo piccolo

Cosa va nel TEST

- Simula dati nuovi mai visti
- Usato solo per la valutazione finale
- Viene scalato con i parametri del train
- Non usato in nessuna fase del training



Data Leakage — L'errore da evitare

Il data leakage avviene quando informazioni del test set filtrano nel training: il modello sembra buono ma in produzione fallisce.

✗ Approccio SBAGLIATO

```
prep = Preprocessor(df)
prep.standardize() ← fit su TUTTO
```

Lo scaler calcola μ e σ su TUTTO il dataset, incluso il test set.

Il modello ha già "visto" i dati futuri
→ metriche ottimistiche e non reali
→ in produzione le performance calano

✓ Approccio CORRETTO

```
X_train, X_test = split(...)
scaler.fit_transform(X_train) ← solo train
scaler.transform(X_test)
```

μ e σ calcolati SOLO sul train.

Il test viene trasformato applicando quei parametri — i dati del test non hanno mai influenzato lo scaler.
→ valutazione realistica e onesta

Standardizzazione — Z-Score Scaling

Le feature hanno scale diverse: weight va da 1000 a 5000 libbre, cylinders da 3 a 8. Senza scaling il modello pesa troppo le feature con valori grandi.

$$X_{\text{scaled}} = (X - \mu) / \sigma$$

μ

Media calcolata sul train set

σ

Deviazione standard del train set

risultato

Media ≈ 0 e std ≈ 1

Esempio pratico — feature weight

```
Valore originale : weight = 3504 lbs
Media train  (μ) : 2977
Std train    (σ) : 849
Risultato     : (3504 - 2977) / 849 = 0.62
```

Dopo scaling:
tutte le feature
hanno media ≈ 0
e std ≈ 1

Il Modello — Regressione Lineare

La regressione lineare cerca i coefficienti che minimizzano l'errore quadratico tra valori reali e previsti.

$$\text{mpg} = \beta_0 + \beta_1 \cdot \text{cyl} + \beta_2 \cdot \text{disp} + \beta_3 \cdot \text{hp} + \beta_4 \cdot \text{weight} + \beta_5 \cdot \text{acc} + \beta_6 \cdot \text{year} + \beta_7 \cdot \text{origin}$$

β_0 — Intercetta

Valore di mpg quando tutte le feature scalate sono 0. Nel progetto: 23.61 mpg

$\beta_1 \dots \beta_7$ — Coefficienti

Quanto cambia mpg se quella feature aumenta di 1 unità standardizzata. Ottimizzati durante training

OLS — Minimi Quadrati

L'algoritmo trova i β che minimizzano la somma dei quadrati degli errori: $\sum (y - \hat{y})^2$

Assunzione chiave

Relazione lineare tra feature e target. Su Auto MPG funziona bene ($R^2 = 0.847$) — la linearità regge

Regolarizzazione — Ridge e Lasso

Varianti della regressione lineare che aggiungono una penalità sui coefficienti per evitare overfitting.

Linear

$$\min \Sigma(y - \hat{y})^2$$

Nessuna penalità

Può overfittare su dataset rumorosi o con tante feature.

Ridge (L2)

$$\min \Sigma(y - \hat{y})^2 + \alpha \cdot \Sigma \beta^2$$

Penalizza grandi coeff.

Li riduce ma non li azzerà. Buono quando tutte le feature sono utili.

α (alpha)
piccolo $\rightarrow$ leggera penalità
grande $\rightarrow$ coeff. Prossimi allo 0

Lasso (L1)

$$\min \Sigma(y - \hat{y})^2 + \alpha \cdot \Sigma |\beta|$$

Azzerà coeff. inutili

Fa feature selection automatica. Utile se molte feature sono ridondanti.

α (alpha)
piccolo $\rightarrow$ leggera penalità
grande $\rightarrow$ coeff. azzerati

Metriche di Valutazione

MSE

8.20

$$\sum (y - \hat{y})^2 / n$$

Mean Squared Error

Errore quadratico medio. Penalizza molto gli errori grandi. L'unità è mpg^2 — poco intuitiva da sola.

RMSE

2.86 mpg

$$\sqrt{\text{MSE}}$$

Root Mean Squared Error

Radice dell'MSE. Unità in mpg: in media il modello sbaglia di ± 2.86 mpg sul range 9–46 mpg.

R²

0.847

$$1 - \text{SSres} / \text{SStot}$$

Coefficiente di Determinazione

Spiega l'84.7% della variabilità di mpg. Vicino a 1 = meglio. Cresce aggiungendo feature (anche inutili).

R²adj

0.832

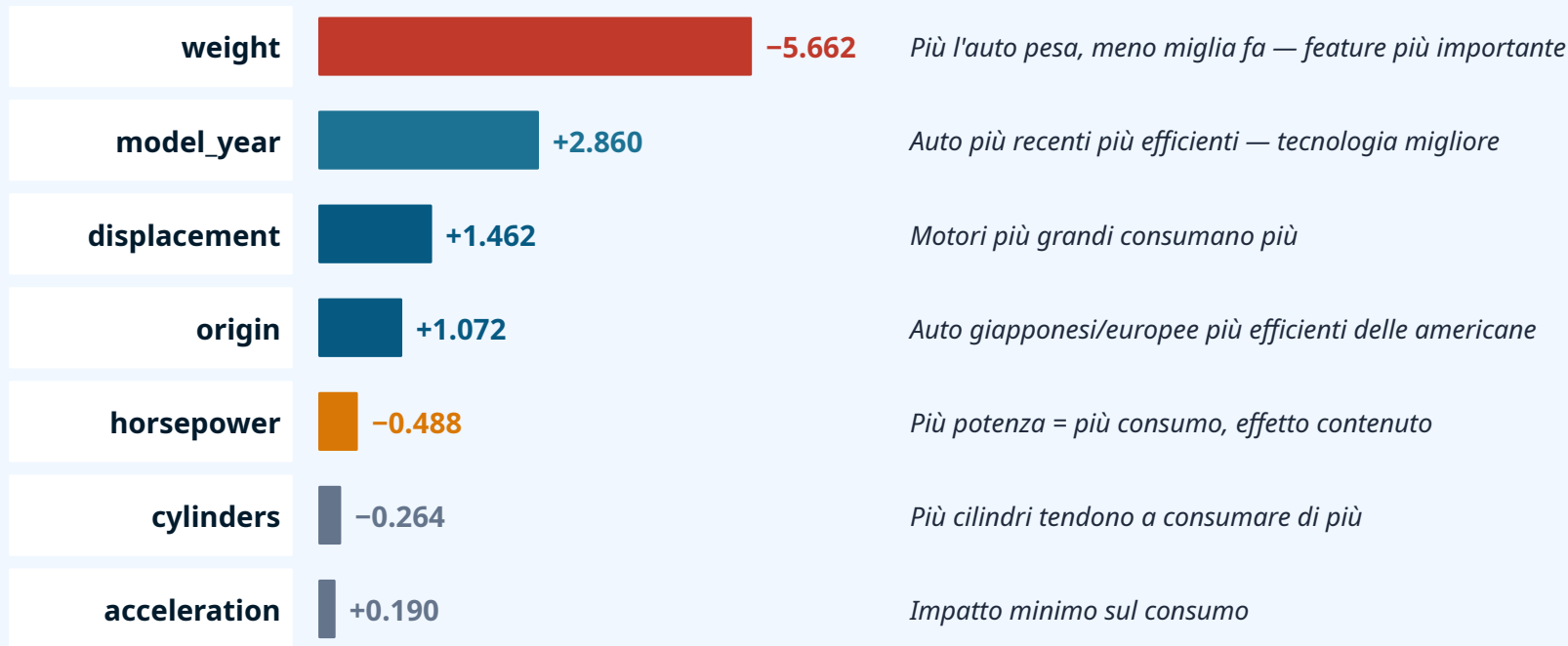
$$1 - (1 - R^2) \cdot (n - 1) / (n - k - 1)$$

R² Aggiustato

Come R² ma penalizza feature inutili. Vicino a R² = le 7 feature sono tutte utili.

Coefficienti — Importanza delle Feature

Ogni coefficiente indica quanto cambia mpg se quella feature aumenta di 1 deviazione standard (dati standardizzati).



Intercetta: 23.61 mpg (valore base quando tutte le feature scalate = 0)

Flask API — Architettura REST

Ogni modulo Python esposto come endpoint HTTP. Il client manda una richiesta, Flask esegue il modulo e risponde con JSON (o PNG per i grafici).

Pipeline	
POST	/load
Scarica il dataset da UCI	
POST	/clean
Gestisce valori mancanti con mediana	
POST	/select
Variance threshold (body: {p: 0.75})	
POST	/pipeline
Esegue tutti gli step in sequenza	

EDA	
GET	/eda/info
Statistiche descrittive del dataset	
GET	/eda/outliers/<col>
Outlier con metodo IQR	
GET	/eda/qqplot/<col>
QQ-plot come immagine PNG	
GET	/eda/histograms
Istogrammi di tutte le feature	

Modello	
POST	/model/train
Addestra linear / ridge / lasso	
POST	/model/predict
Predice mpg da feature in input	
GET	/model/coefficients
Restituisce pesi e intercetta	
GET	/health
Verifica che il server sia attivo	

Docker & Conclusioni

Docker

```
docker build -t auto-mpg .  
docker run -p 5000:5000 \  
  --name auto-mpg-flask auto-mpg
```

*-p 5000:5000 mappa la porta
del container sul localhost*

github.com/Amorlot/auto-mpg

Risultati & Takeaway

$R^2 = 0.847$

Spiega l'84.7% della variabilità

RMSE = 2.86

Errore medio ± 2.86 mpg

weight

Feature più importante (-5.66)

No leakage

Split prima dello scaling

REST API

11 endpoint Flask in Docker

Modularità

Ogni classe ha una sola responsabilità